

Programming for AI

AI-2012

**Introduction to Object-Oriented
Programming | Objects and
Classes**

National University of Computer & Emerging Sciences – NUCES – Karachi



National University of Computer & Emerging Sciences – NUCES – Karachi

Course Code: AI-2001	Programming for AI
----------------------	--------------------

Contents

1. Introduction	2
2. What is Object-Oriented Programming?	2
3. Objects and Classes	2
4. Specifying Attributes & Behaviours	3
4.1 Data Describes Object	3
4.2. Behaviours are Actions	3
5. Creating Classes	3
5.1. Adding Attributes	3
5.2. Making the above class do something.....	4
5.3. The self Argument.....	5
5.3. Multiple Arguments in a Method	5

1. Introduction

An **object** is a collection of **data** and **behaviours**. Object-oriented refers to the approach of directing software development toward modelling these **objects**. It is a technique used to model complex systems by representing interacting **objects** through their **data** and **behaviours**.

Terms like **object-oriented analysis, design, and programming** all fall under the broader concept of **object-oriented development**, representing different stages of the software creation process. Referring to these stages as object-oriented indicates the specific style of development being used.

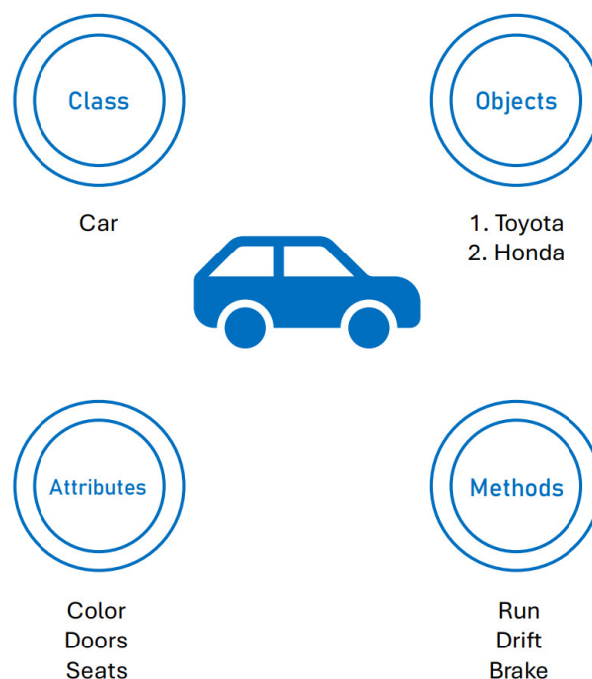
2. What is Object-Oriented Programming?

Object-Oriented Programming (OOP) is a way of writing programs that revolves around "objects." An **object** can be anything that has **data (information)** and **actions (things it can do)**. Instead of writing a program as one big list of instructions, OOP helps you organize your code by grouping related information and actions together.

3. Objects and Classes

An object is an actual thing created using the **class**. It's like using the blueprint (**class**) to build a specific car. Each car you create using the **Car** class is an object, with its own unique details.

A **class** is like a blueprint or template. It defines what an object will look like and what it can do. For example, if you want to create multiple cars in your program, you first define a **class** called **Car**. This **class** will describe what properties (like **colour, model**) and actions (like **driving**) a car should have.



4. Specifying Attributes & Behaviours

In object-oriented programming, objects are specific instances of classes, each with its own set of **data (attributes)** and **behaviours (actions)**. For example, a **Car** class might define characteristics like **colour** and **model**, shared by all cars. However, each car object can have its own unique values for these attributes.

4.1 Data Describes Object

Data describes the individual characteristics of an object. For instance, if you have two cars (objects) created from the **Car** class, one might be **red** and another **blue**. Both cars share the same attributes, like **colour**, but have different values for them. Attributes, also known as **members** or **properties**, represent the data related to each object. Some authors make a distinction by saying that attributes can be changed, while properties are read-only.

For example, the **Car** class could have a **colour** attribute, and each specific car (like **car1** and **car2**) would have its own **colour** value, such as **red** or **blue**. Though the colour attribute is shared, the values can differ between objects.

4.2. Behaviours are Actions

Behaviours are the actions that an object can perform, and in object-oriented programming, these are called methods. Methods are similar to functions in structured programming but have special access to the data of the object they belong to. Like functions, methods can take inputs (called parameters or arguments) and can return results.

5. Creating Classes

We don't need to write much Python code to notice that it's a very "clean" language. When we want to accomplish something, we can just do it without much setup. For example, the classic **"hello world"** program in Python is just a single line. Similarly, the most basic class in Python is as simple as:

```
class MyFirstClass:  
    pass
```

This is our first object-oriented program! The class definition begins with the **class** keyword, followed by a name (which we choose to identify the class), and ends with a colon.

Because our first class doesn't perform any actions, we use the **pass** keyword on the second line to indicate that no additional code is needed.

5.1. Adding Attributes

Now we have a simple class, but it's not very useful since it doesn't hold any data or perform any actions. So, how do we assign an attribute to an object?

Interestingly, we don't need to modify the class definition for this. We can assign attributes to an object after it's created by using dot notation.

```

class Point:
    pass

p1 = Point()
p2 = Point()

p1.x = 5
p1.y = 4
p2.x = 3
p2.y = 6

print(p1.x, p1.y)
print(p2.x, p2.y)

```

Output:	5 4 3 6
---------	------------

This code defines an empty **Point** class with no data or functionality. It then creates two instances of the class and assigns **x** and **y** coordinates to each, representing points in a 2D space. To assign values to an object's attributes, we use the **<object>.<attribute> = <value>** syntax, commonly called dot notation. The assigned value can be anything, such as a Python primitive, a built-in data type, another object, or even a function or class.

5.2. Making the above class do something

Having objects with attributes is useful, but the core of object-oriented programming is how objects interact. We want to perform actions that affect those attributes. Now it's time to add behaviours to our classes. Let's model a couple of actions for our **Point** class, starting with a method called **reset**, which moves the point to the origin (where both **x** and **y** are zero). This is a good first example since it doesn't need any parameters.

```

class Point:
    def reset(self):
        self.x = 0
        self.y = 0

p = Point()

p.reset()

print(p.x, p.y)

```

Output:	0 0
---------	-----

5.3. The self Argument

The main difference between **methods** and regular **functions** is that methods require one additional **argument**. By convention, this argument is named **self**, though technically, you could call it anything, even something like **this** or **Martha**.

The **self** argument is simply a reference to the object on which the method is called. Through **self**, you can access the attributes and methods of the object, just like you would with any other object. For example, in the **reset** method, we use **self** to modify the object's **x** and **y** attributes.

When we call the **p.reset()** method, we don't need to provide the **self** argument. Python automatically handles this, as it knows we're calling the method on the **p** object and passes that object to the method behind the scenes.

However, the method is essentially a function within a class. Instead of calling it on the object, we can also call the function directly from the class and explicitly pass the object as the **self** argument.

```
p = Point()
Point.reset(p)
print(p.x, p.y)
```

What happens if we forget to include the **self** argument in our class definition? Python will bail with an error message:

```
class Point:
    def reset(self):
        self.x = 0
        self.y = 0

p = Point()

p.reset()

print(p.x, p.y)
```

Error:	Traceback (most recent call last): File "oop.py", line 7, in <module> p.reset() TypeError: Point.reset() takes 0 positional arguments but 1 was given
---------------	--

5.3. Multiple Arguments in a Method

How do we pass multiple arguments to a method? Let's introduce a new method that lets us move a point to any position, not just the origin. Additionally, we can create a method that takes another **Point** object as input and calculates the distance between the two points.

```

import math
class Point:
    def move(self, x, y):
        self.x = x
        self.y = y

    def reset(self):
        self.move(0, 0)

    def calculate_distance(self, other_point):
        return math.sqrt((self.x - other_point.x)**2 +(self.y -
other_point.y)**2)
# how to use it:

point1 = Point()
point2 = Point()

point1.reset()
point2.move(5,0)

print(point2.calculate_distance(point1))

assert (point2.calculate_distance(point1) == point1.calculate_distance(point2))

point1.move(3,4)

print(point1.calculate_distance(point2))
print(point1.calculate_distance(point1))

```

	5.0
Output:	4.47213595499958
	0.0

A lot has changed in this code. The class now includes three methods. The **move** method takes two arguments, **x** and **y**, and assigns those values to the **self** object, similar to how the previous **reset** method worked. The updated **reset** method now calls **move**, as resetting a point essentially means moving it to a specific known position.

The **calculate_distance** method employs the Pythagorean theorem to find the distance between two points. While understanding the math involved (where ****** represents squaring a number, and **math.sqrt** computes the square root) is helpful, it isn't essential for our current goal of learning how to write methods.

The sample code at the end of the previous example illustrates how to invoke a method with arguments: you simply place the arguments within the parentheses and use dot notation to access the method. I chose some arbitrary positions to test the methods. The test code calls each method and prints the results to the console. The **assert** function acts as a basic testing tool; if the statement following **assert** evaluates to **False** (or is **zero**, **empty**, or **None**), the program will terminate. In this instance, we use it to verify that the distance remains consistent regardless of which point calls the other point's **calculate_distance** method.